

Eksamen IN1010

4. juni 2025

Generelt om oppgaven

- I dette oppgavesettet skal du *ikke* bruke arrayer, ArrayList-er, HashMap-er eller andre av Java sine innebygde beholdere. I stedet skal du definere og bruke en egen datastruktur kalt **Samling** (som er beskrevet i første oppgave).
- Du kan innføre flere parametre, variabler og metoder enn de som er omtalt i oppgaveteksten. Noen av oppgavene krever utvidelse av klasser skrevet i tidligere oppgaver. Beskriv hvor utvidelsen hører til i en kommentar.
- Du kan gjenbruke kode fra andre deloppgaver selv om du ikke har løst oppgaven. Anta i så fall at omtalte klasser og metoder finnes.
- Hvis du synes noe i oppgaveteksten er uklart, kan du gjøre dine egne antagelser. Skriv en kommentar om dette i svaret ditt.

Oppgave 1 Klassen Samling (vekt 10 poeng)

Klassen Samling skal deklarerer slik:

```
class Samling<E> {
    private Node start;

    public Samling (int n) { ... }

    private class Node {
        ...
    }

    public void settInn (E ny) { ... }
    public void fjern (E ref) {
        /* skal ikke skrives, du kan anta
           at den er implementert */
    }
}
```

Konstruktøren skal opprette en lenkeliste med n noder. Etter at konstruktøren er ferdig vil det aldri bli laget nye noder eller fjernet noen. Alle noderes datareferanse skal initieres til **null** av konstruktøren. Generelt gjelder at om en datareferanse er **null**, skal det angi at det ikke er noe element tilknyttet noden akkurat nå.

Metoden `settInn` skal sette inn et nytt element i en ledig node, dvs en node som har datareferanse **null**. Det er det samme hvilken node som velges. Du kan anta at det alltid finnes en ledig node.

Metoden `fjern` fjerner det gitte elementet fra listen ved å sette dets datareferanse til **null**. Denne metoden skal du *ikke* skrive.

(I den virkelige verden ville klassen hatt mange flere metoder, men i denne eksamensoppgaven er dette tilstrekkelig.)

Restauranten har en `Samling` med inntil 25 ansatte, og hver kelner har en `Samling` av inntil 10 bord som hen betjener.

Til restauranten kommer det grupper med gjester; det kan være inntil 20 gjester i hver gruppe. Den enkelte gjest opplyser om vedkommende ønsker vin til maten (angitt som parameter til konstruktøren til gjesten).

Skriv klassene `Restaurant`, `Ansatt`, `Kelner`, `Vinkelner`, `Lærling`, `Bord`, `Gjest` og `Gjestegruppe` med instansvariabler og konstruktører, men ikke andre metoder.

Oppgave 5 Søking (vekt 8 poeng)

Utvid klassen `Gjest` med metoden `boolean ønskerVin()`.

Utvid deretter klassen `Gjestegruppe` med metoden `boolean noenØnskerVin()` som finner ut om minst én i gruppen ønsker vin. Metoden *skal* benytte iteratoren fra oppgave 2 til å sjekke gjestene.

Oppgave 6 Finn beste (vekt 8 poeng)

Skriv metoden `Kelner finnMinstÅGjøre(boolean erVinkelner)` i klassen `Restaurant` som finner den kelneren som betjener færrest antall gjester. (NB! færrest antall gjester, ikke færrest antall bord.) Hvis `erVinkelner` er sann, er bare `Vinkelner`-e interessante. Vi kan anta at det alltid er minst én som fyller kriteriene.

Oppgave 7 Rekursjon (vekt 10 poeng)

Utvid klassen `Samling` fra oppgave 1 med en metode `antall` som finner antall elementer. Dette skal gjøres ved å gå gjennom samlingen *rekursivt*.

Oppgave 8 Ta imot gjester (vekt 12 poeng)

Skriv metoden `taImotGjester(Gjestegruppe gruppe)` i klassen `Restaurant`. Den skal gjøre følgende:

1. Finn et ledig bord med nok plasser. (Du kan her anta at det alltid finnes et slikt bord.)
2. Finn den kelneren som for tiden har minst å gjøre. Ta hensyn til om noen av gjestene i gruppen ønsker vin.
3. Oppdater referansene mellom gjestegruppen, kelneren og bordet.

Gjør bruk av metoder definert tidligere, og tilføy eventuelle hjelpemetoder i andre klasser ved behov.

Oppgave 9 GUI (vekt 10 poeng)

Endre metoden `taImotGjester` i klassen `Restaurant` slik at hvis det ikke finnes noe ledig bord med nok plass til gruppen av gjester, skal programmet lage et GUI-vindu med teksten "Beklager, restauranten er full!", samt en OK-knapp som lukker vinduet. Vinduet skal vises på skjermen og kan se slik ut:



Oppgave 10 Tråder (vekt 14 poeng)

I denne oppgaven skal du implementere venting på ledig bord ved hjelp av tråder. Du skal ta utgangspunkt i koden du hadde *før* du gjorde endringen i `taImotGjester` i forrige oppgave.

Endre klassen `Restaurant` så den blir en monitor. Metoden `taImotGjester` skal nå fungere slik: Hvis det ikke er noen ledige bord som har nok plasser, må kallet vente på en `Condition` til et slikt bord blir ledig. Utvid også klassen `Restaurant` med en ny metode `forlatBordet(Bord bord)` som registrerer at bordet er ledig og signalerer ventende gjestegrupper.

Skriv også trådklassen `Restaurantbesøk` der konstruktøren skal ha to parametre: `Restaurant restaurant` og `Gjestegruppe gruppe`. `run`-metoden skal gjøre dette:

1. Kalle metoden `restaurant.taImotGjester`. (Siden `Restaurant` nå er en monitor, vil dette kallet ikke returnere før det finnes et ledig bord.)
2. Spise (men dette skal du ikke programmere)
3. Kalle metoden `restaurant.forlatBordet`.